

LAB SESSION 11

QUICK SORT

Introduction

Quick Sort is one of the most efficient and widely used sorting algorithms in computer science. It follows the divide and conquers technique, where a large problem is divided into smaller sub-problems, solved individually, and then combined to obtain the final solution.

In Quick Sort, a pivot element is selected from the array. The array is then partitioned in such a way that all elements smaller than the pivot are placed on the left side and all elements greater than the pivot are placed on the right side. After placing the pivot in its correct position, the same process is recursively applied to the left and right subarrays until the entire array becomes sorted.

Quick Sort is known for its high efficiency and is commonly used in practical applications because of its good average-case performance. The average time complexity of Quick Sort is $O(n \log n)$, although in the worst case it can be $O(n^2)$.

This lab experiment aims to implement the Quick Sort algorithm using a programming language, understand the partitioning process, and analyze its working through program execution and output.

Algorithm: Quick Sort

Input: Array A with n elements

Output: Sorted array

1. Start
2. Select a **pivot element** from the array (usually the last element).
3. Partition the array so that:
 - Elements **smaller than pivot** go to the left.
 - Elements **greater than pivot** go to the right.
4. Place the pivot in its **correct sorted position**.
5. Recursively apply **Quick Sort** to the left subarray.
6. Recursively apply **Quick Sort** to the right subarray.
7. Stop when the subarray has **0 or 1 element**.
8. End.

Time Complexity

- **Best Case:** $O(n \log n)$
- **Average Case:** $O(n \log n)$
- **Worst Case:** $O(n^2)$

Code

```
#include <iostream>

#include <vector>

using namespace std;

// Function to partition the array
// It places the pivot element at its correct position
int partition(vector<int> &array, int low, int high) {
    int pivot = array[high];    // Choose the last element as pivot
    int i = low - 1;           // Index of smaller element
    // Traverse elements from low to high-1
    for (int j = low; j < high; j++) {
        // If current element is smaller than or equal to pivot
        if (array[j] <= pivot) {
            i++;                // Move index of smaller element forward
            swap(array[i], array[j]); // Swap current element with element at i
        }
    }
    // Place pivot at the correct position in sorted array
    swap(array[i + 1], array[high]);
    // Return the pivot index
    return i + 1;
}

// Quick Sort function
void quicksort(vector<int> &array, int low, int high) {
```

```

// Check if there are at least two elements
if (low < high) {
    // Partition the array and get pivot index
    int pivot_index = partition(array, low, high);
    // Recursively sort elements before pivot
    quicksort(array, low, pivot_index - 1);
    // Recursively sort elements after pivot
    quicksort(array, pivot_index + 1, high);
}
}

int main() {
    // Initialize the array
    vector<int> my_array = {64, 34, 25, 12, 22, 11, 90, 5};
    // Call quicksort function
    quicksort(my_array, 0, my_array.size() - 1);
    // Print sorted array
    cout << "Sorted array: ";
    for (int num : my_array) {
        cout << num << " ";
    }

    return 0;
}

```

Tasks:

1. Write a program to implement the Quick Sort algorithm to sort an array of integers.

Description:

- Create an array of unsorted numbers.
- Apply the Quick Sort algorithm.
- Display the array **before sorting** and after sorting.

Example Input:

64, 34, 25, 12, 22, 11, 90, 5

Expected Output:

5, 11, 12, 22, 25, 34, 64, 90

2. Write a program that allows the user to enter elements and sorts them using Quick Sort.

Description:

- Ask the user to enter the **number of elements**.
- Take the array elements as input.
- Use Quick Sort to arrange them in **ascending order**.
- Display the sorted array.